Practical Experiment Worksheet–6

Understanding and Implementing Cursors in PL/SQL

---

1. Aim of the Experiment

To understand the concept and working of **implicit and explicit cursors in PL/SQL** for row-by-row data processing, and to apply business logic using cursor attributes.

---

2. Objectives of the Experiment

The objectives of this experiment are:

1. To understand the concept of cursors in PL/SQL.
2. To differentiate between implicit and explicit cursors.
3. To learn how to process multiple rows using explicit cursors.
4. To use cursor attributes such as %FOUND, %NOTFOUND, %ROWCOUNT, and %ISOPEN.
5. To implement business logic on database records using cursors.
6. To analyze row-by-row data processing in real-world scenarios.

---

3. Software / Tools Required

- Oracle Database Express Edition (Oracle XE)
- Oracle SQL Developer / SQL*Plus
- Computer system with database access

---

4. Theory

🔹 PL/SQL and Cursors

PL/SQL is Oracle's procedural extension of SQL that allows developers to write programs with loops, conditions, and logic.

When a SQL query returns multiple rows, PL/SQL uses **cursors** to process each row individually.

🔹 Types of Cursors

1. Implicit Cursor

- Automatically created by Oracle

- Used for single-row operations (INSERT, UPDATE, DELETE)
- No declaration required

Example:

- UPDATE statement

---

2. Explicit Cursor

- Defined by the programmer
- Used when query returns multiple rows
- Requires:
    - Declaration
    - Opening
    - Fetching
    - Closing

🔹 **Cursor Attributes**

Attribute	Description
%FOUND	TRUE if row is fetched
%NOTFOUND	TRUE if no row is fetched
%ROWCOUNT	Number of rows processed
%ISOPEN	Checks if cursor is open

🔹 **Need of Cursors**

- Required when **row-by-row processing** is needed
- Used in business logic like:
    - Salary updates
    - Bonus calculation
    - Record-wise validation

---

5. Procedure

Step 1
Start the system and open Oracle SQL Developer.

Step 2
Open a new SQL Worksheet.

Step 3
Drop existing table (if exists).

Step 4
Create Employee table.

Step 5
Insert records into table.

Step 6
Execute implicit cursor program.

Step 7
Execute explicit cursor program.

Step 8
Apply business logic using cursor.

Step 9
Use cursor attributes and observe output.

---

6. SQL / PL-SQL Commands Used

Step 1: Drop Table

```
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE Employee';
EXCEPTION
   WHEN OTHERS THEN NULL;
END;
/
```

Step 2: Create Table

```
CREATE TABLE Employee (
   EmpID NUMBER PRIMARY KEY,
   EmpName VARCHAR2(50),
   Salary NUMBER,
   DeptID NUMBER
);
```

Step 3: Insert Data

```
INSERT INTO Employee VALUES (1, 'Aman', 30000, 10);
INSERT INTO Employee VALUES (2, 'Riya', 40000, 20);
INSERT INTO Employee VALUES (3, 'Karan', 50000, 10);
INSERT INTO Employee VALUES (4, 'Neha', 35000, 30);

COMMIT;
```

Step 4: Implicit Cursor Example

```
BEGIN
   UPDATE Employee
   SET Salary = Salary * 1.10
   WHERE DeptID = 10;

   IF SQL%FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Rows Updated: ' || SQL%ROWCOUNT);
   ELSE
      DBMS_OUTPUT.PUT_LINE('No rows updated');
   END IF;
END;
/
```

Step 5: Explicit Cursor Example

```
DECLARE
   CURSOR emp_cursor IS
      SELECT EmpID, EmpName, Salary FROM Employee;

   v_id Employee.EmpID%TYPE;
   v_name Employee.EmpName%TYPE;
   v_salary Employee.Salary%TYPE;

BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO v_id, v_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE('ID: ' || v_id ||
                           ' Name: ' || v_name ||
                           ' Salary: ' || v_salary);
   END LOOP;
```

```
   CLOSE emp_cursor;
END;
/
```

Step 6: Cursor with Business Logic

```
DECLARE
   CURSOR emp_cursor IS
      SELECT EmpID, Salary FROM Employee;

   v_id Employee.EmpID%TYPE;
   v_salary Employee.Salary%TYPE;

BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO v_id, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;

      IF v_salary < 40000 THEN
         UPDATE Employee
         SET Salary = Salary + 5000
         WHERE EmpID = v_id;

         DBMS_OUTPUT.PUT_LINE('Bonus added to EmpID: ' || v_id);
      END IF;

   END LOOP;

   CLOSE emp_cursor;
END;
/
```

Step 7: Cursor Attribute Example

```
DECLARE
   CURSOR emp_cursor IS SELECT * FROM Employee;
BEGIN
   OPEN emp_cursor;

   IF emp_cursor%ISOPEN THEN
      DBMS_OUTPUT.PUT_LINE('Cursor is open');
   END IF;
```

```
    CLOSE emp_cursor;
END;
/
```

---

7. Output

Expected output includes:

- Rows Updated: 2
- Employee details displayed row by row
- Bonus added messages
- Cursor open confirmation

```
SQL> BEGIN
         UPDATE Employee
         SET Salary = Salary * 1.10
         WHERE DeptID = 10;...
Show more...


Rows Updated: 2


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.100
```

```
SQL> DECLARE
         CURSOR emp_cursor IS
             SELECT EmpID, EmpName, Salary FROM Employee;
     ...
Show more...


ID: 1 Name: Aman Salary: 33000
ID: 2 Name: Riya Salary: 40000
ID: 3 Name: Karan Salary: 55000
```

```
SQL> DECLARE
         CURSOR emp_cursor IS
             SELECT EmpID, Salary FROM Employee;
     ...
Show more...


Bonus added to EmpID: 1
Bonus added to EmpID: 4


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006
```

```
SQL> DECLARE
         CURSOR emp_cursor IS SELECT * FROM Employee;
     BEGIN
         OPEN emp_cursor;...
Show more...


Cursor is open
```

8. Observations

1. The Employee table was successfully created.
2. Records were inserted correctly.
3. Implicit cursor updated multiple rows efficiently.
4. Explicit cursor processed rows one by one.
5. Cursor attributes controlled execution flow effectively.
6. Business logic was applied successfully using cursors.

9. Result

The experiment was successfully performed.
Cursors were implemented to process multiple rows, and PL/SQL programs were used to apply business logic using implicit and explicit cursors.